LaunchPad AI: AI-Powered Startup Viability & Strategy Engine

Project Description:

LaunchPad AI harnesses Generative AI to help founders quickly evaluate and refine their startup ideas. Users describe their startup name, target audience, description, and business model, and the platform returns a structured viability assessment. The system generates a **Viability Score** and **Innovation Score** (0-100), a full **SWOT Report** covering strengths, weaknesses, opportunities, and threats, and a **Recommendations** section containing market insights and an actionable growth strategy — all from a single form submission.

Utilizing Google's Gemini API with the gemini-3.5-flash model, LaunchPad AI processes user input through a tightly-constrained, JSON-only prompt to deliver consistent, structured startup analysis. Built with Flask and Flask-SQLAlchemy, the platform persists every generated report to a local SQLite database so founders can revisit past analyses under "Recent Reports." The app is deployed publicly via Ngrok, with secure API key management through a .env file and strict input validation (2000-character limits on all fields) to keep the experience reliable and abuse-resistant.

Scenarios

Scenario 1: A founder describes "Diet Plan Ai," an AI-powered platform that generates personalized daily nutrition plans, targeting students, elderly people, and busy professionals with a freemium business model. LaunchPad AI returns a Viability Score of 55 and Innovation Score of 42, flags an overly broad target audience as a key weakness, and recommends narrowing the launch focus to busy professionals with grocery delivery integration as a growth strategy.

Scenario 2: A founder describes a B2B SaaS tool for automating expense reports, targeting small accounting firms with a subscription pricing model. LaunchPad AI generates a higher viability score given the validated market pain point, identifies churn risk from feature parity with QuickBooks as a threat, and recommends a niche vertical focus (e.g., restaurants or construction) as a differentiation strategy.

Scenario 3: A founder describes a marketplace connecting local artisans with buyers, targeting Gen Z consumers with a commission-based model. LaunchPad AI's SWOT report highlights strong community appeal as a strength but flags thin margins from commission-only revenue as a weakness, recommending a hybrid subscription-plus-commission model in the growth strategy.

Technical Architecture:

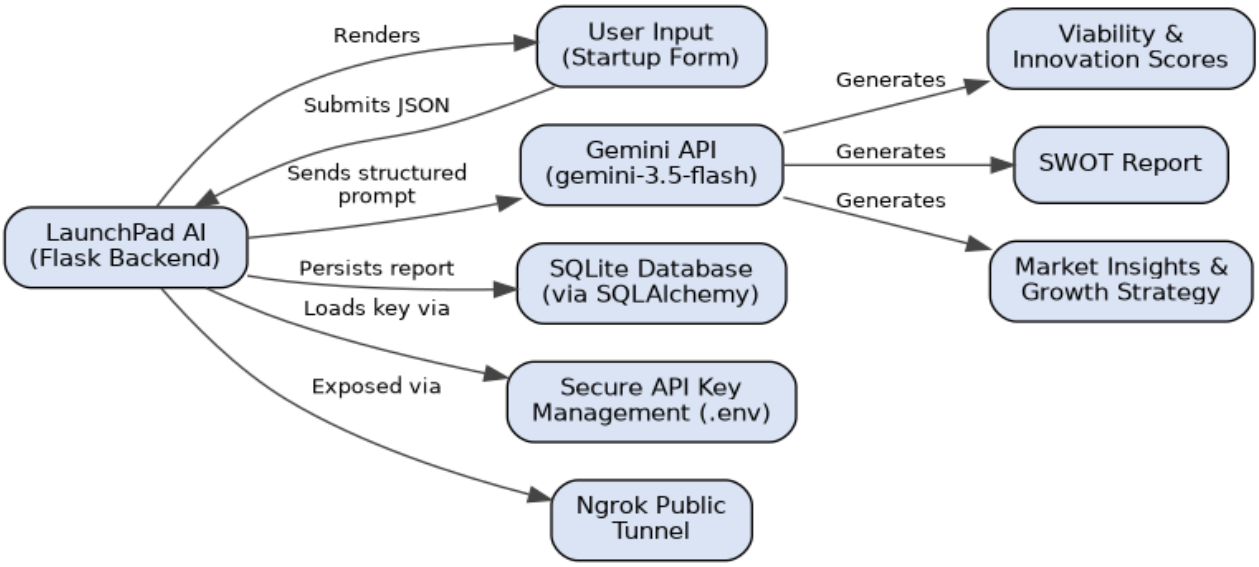

Fig 1: LaunchPad AI system architecture

Description: LaunchPad AI follows a modular three-tier architecture. The frontend (HTML, CSS, JavaScript) renders the startup input form and results dashboard. The Flask backend validates incoming requests, constructs a structured JSON-only prompt, and orchestrates the call to the Gemini API (gemini-3.5-flash). Generated reports are persisted to a local SQLite database via Flask-SQLAlchemy, the API key is managed securely through a .env file loaded with python-dotenv, and the local server is exposed publicly through an Ngrok tunnel.

Database Schema (ER Diagram):

LaunchPad AI uses a single-table schema (**startup_reports**) to persist every generated analysis:

Column	Type	Description
id	Integer (PK)	Auto-incrementing primary key
startup_name	String(200)	Name of the startup being analyzed
description	Text	User-provided startup description
target_audience	Text	User-provided target audience
business_model	Text	User-provided business model
viability_score	Integer	AI-generated viability score (0-100)
innovation_score	Integer	AI-generated innovation score (0-100)
swot_json	Text	SWOT report stored as a serialized JSON string
market_insights	Text	AI-generated market analysis
growth_strategy	Text	AI-generated growth recommendations
created_at	DateTime	Timestamp the report was generated

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Gemini API Familiarity:** Google AI Studio / Gemini API Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Flask-SQLAlchemy / SQLite:** Flask-SQLAlchemy Documentation
- **Version Control with Git:** Git Documentation
- **Ngrok for Public Deployment:** Ngrok Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

Milestone 2: Core Functionalities Development

Milestone 3: App.py Development

Milestone 4: Frontend Development

Milestone 5: Deployment

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini model for structured startup analysis.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, database, and AI integration.
- **Activity 1.4:** Set up the development environment, installing Flask, Flask-SQLAlchemy, the Gemini SDK, pyngrok, and python-dotenv.

Milestone 1: Model Selection and Architecture

This milestone focuses on selecting the right Gemini model for structured startup analysis and defining how the frontend, backend, database, and AI integration fit together.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with Gemini, a valid API key is required.

- **Step 1 — Create a Google AI Studio Account:** Visit https://aistudio.google.com and sign in with a Google account.
- **Step 2 — Get an API Key:** Navigate to "Get API Key" and create a new key for the project.
- **Step 3 — Add the Key to the Project:** Create a .env file in the project root and add:

```
GEMINI_API_KEY=your_gemini_api_key_here
NGROK_AUTHTOKEN=your_ngrok_authtoken_here
PORT=5000
```

Step 4 — Verify the Key is Loaded: app.py loads environment variables at startup via python-dotenv, and ai_engine.py reads the key when configuring the Gemini client:

```
load_dotenv()

def _configure_client():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key or api_key == "your_gemini_api_key_here":
        raise AIEngineError(
            "GEMINI_API_KEY is missing or not set..."
        )
    genai.configure(api_key=api_key)
```

Important — Never Commit Your API Key: Add .env to .gitignore to prevent accidentally pushing the key to a public repository.

Activity 1.2: Research and Select the Appropriate Model

- **Understand the Requirements:** LaunchPad AI needs structured, strictly-formatted JSON output (scores, SWOT arrays, free-text insights) rather than open-ended chat text.
- **Explore Gemini's Model Documentation:** Compare available Gemini models on latency, context window, and JSON-mode support.
- **Evaluate Performance:** Prioritize models that support a native response_mime_type: "application/json" generation config to reduce parsing failures.
- **Select the Optimal Model:** Choose **gemini-3.5-flash** for its balance of speed and reasoning quality, set with a temperature of 0.7 and JSON response mode enabled.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** See Fig. 1 — frontend, Flask backend, SQLite database, and Gemini API integration.
- **Detail Frontend Functionality:** Users enter startup name, target audience, description, and business model, then click "Launch Analysis."
- **Outline Backend Responsibilities:** The Flask backend handles POST requests to /api/analyze, validates and sanitizes input via validate_input(), and calls generate_startup_analysis().

- **Describe AI Integration Points:** The backend builds a structured prompt via _build_prompt(), calls the Gemini API, parses/validates the JSON via _extract_json() and _validate_payload(), and returns structured results to the frontend and database.

Activity 1.4: Set Up the Development Environment

```
python -m venv env
env\Scripts\activate  (Windows)
source env/bin/activate  (macOS/Linux)

pip install Flask flask-sqlalchemy google-generativeai python-dotenv pyngrok
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

LaunchPad AI generates five core outputs from a single structured Gemini prompt:

- **Viability Score:** An integer 0-100 representing overall business viability, clamped defensively after parsing.
- **Innovation Score:** An integer 0-100 representing novelty and differentiation.
- **SWOT Report:** An object with exactly four keys — strengths, weaknesses, opportunities, threats — each a short array of 3-5 points.
- **Market Insights:** A 3-5 sentence analysis of the target market.
- **Growth Strategy:** A 3-5 sentence set of concrete, actionable growth recommendations.

These required keys are enforced in code rather than left to chance:

```
REQUIRED_KEYS = [
    "viability_score",
    "innovation_score",
    "swot",
    "market_insights",
    "growth_strategy",
]

REQUIRED_SWOT_KEYS = ["strengths", "weaknesses", "opportunities", "threats"]
```

Activity 2.2: Implement the Flask Backend

Validate User Input: validate_input() enforces required fields and a 2000-character limit before any AI call is made:

```
def validate_input(payload: dict):
    required_fields = ["startup_name", "description",
                       "target_audience", "business_model"]
    cleaned = {}
    for field in required_fields:
        value = str(payload.get(field, "") or "").strip()
        if not value:
            return None, f"Field '{field}' cannot be empty."
        if len(value) > MAX_INPUT_LENGTH:
            return None, f"Field '{field}' exceeds the limit."
        cleaned[field] = value
    return cleaned, None
```

Route AJAX Requests: The frontend submits JSON to /api/analyze, which validates input, calls generate_startup_analysis(), persists a StartupReport row, and returns the structured result as JSON (see Milestone 3 for the full route implementation).

Milestone 3: App.py Development

Milestone 3 covers the core logic in app.py — Flask setup, the SQLAlchemy model, route handlers, and the Ngrok tunnel used for public deployment.

Activity 3.1: Flask & Database Setup

```
app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:///{os.path.join(BASE_DIR, 'instance', 'launc
hpad.db')}"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
db = SQLAlchemy(app)

class StartupReport(db.Model):
    __tablename__ = "startup_reports"
    id = db.Column(db.Integer, primary_key=True)
    startup_name = db.Column(db.String(200), nullable=False)
    viability_score = db.Column(db.Integer, nullable=False)
    innovation_score = db.Column(db.Integer, nullable=False)
    swot_json = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

Activity 3.2: Core Routes

***/* — Home route** renders the main dashboard:

```
@app.route("/")
def index():
    return render_template("index.html", max_length=MAX_INPUT_LENGTH)
```

/api/analyze — POST endpoint validates input, calls the AI engine, persists the report, and returns structured JSON:

```
@app.route("/api/analyze", methods=["POST"])
def analyze_startup():
    payload = request.get_json(silent=True)
    cleaned, error = validate_input(payload)
    if error:
        return jsonify({"success": False, "error": error}), 400
    try:
        analysis = generate_startup_analysis(
            name=cleaned["startup_name"],
            description=cleaned["description"],
            audience=cleaned["target_audience"],
            business_model=cleaned["business_model"],
        )
    except AIEngineError as e:
        return jsonify({"success": False, "error": str(e)}), 502
        # ...persist StartupReport, return jsonify({"success": True, "data": report.to_dict()}
)
```

/api/history returns the 20 most recent saved reports; **/api/history/<id>** (DELETE) removes a single saved report. Both are wrapped in try/except blocks that roll back the session and return a clean JSON error on failure.

Milestone 3: App.py Development (continued)

Activity 3.3: Ngrok Tunnel & Entry Point

start_ngrok_tunnel() authenticates with Ngrok using the token from .env and opens a public HTTP tunnel to the local Flask server, printing the public URL to the terminal:

```
def start_ngrok_tunnel(port: int):
    authtoken = os.environ.get("NGROK_AUTHTOKEN")
    if not authtoken or authtoken == "your_ngrok_authtoken_here":
        logger.warning("NGROK_AUTHTOKEN not set — running locally only.")
        return None
    conf.get_default().auth_token = authtoken
    tunnel = ngrok.connect(port, "http")
    logger.info(" LaunchPad AI is LIVE")
    logger.info(" Public: %s", tunnel.public_url)
    return tunnel
```

Important: the app runs with debug=False so Flask's reloader does not spawn a second process, which would open a duplicate, conflicting Ngrok tunnel.

```
if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    PORT = int(os.environ.get("PORT", 5000))
    start_ngrok_tunnel(PORT)
    app.run(host="0.0.0.0", port=PORT, debug=False)
```

Milestone 4: Frontend Development

Milestone 4 covers the user-facing dashboard: a clean input form and a results view that renders scores, SWOT, and recommendations returned by the backend.

Activity 4.1: Designing the Input Interface

- **Header:** "LaunchPad AI" branding with the tagline "Startup Viability & Strategy Engine."
- **Describe Your Startup form:** four fields — Startup Name, Target Audience, Description, and Business Model — each a textarea with a live character counter capped at 2000/2000.
- **Launch Analysis button:** submits the form via Fetch API as a JSON POST to /api/analyze, disabling itself and showing a "Processing your analysis..." indicator while the request is in flight.

Activity 4.2: Rendering Results Dynamically

- **Score Cards:** Two large numeric cards for Viability Score and Innovation Score.
- **SWOT Report:** Four color-coded quadrant cards — Strengths (green), Weaknesses (red), Opportunities (blue), Threats (orange) — each rendering the AI-generated bullet arrays.
- **Recommendations:** A Market Insights paragraph and a Growth Strategy paragraph rendered directly beneath the SWOT grid.
- **Recent Reports:** A running list of previously generated reports (name, viability, and innovation score) each with a "View" button, backed by GET /api/history.

Milestone 5: Deployment

Milestone 5 covers deploying LaunchPad AI first locally, then publicly via Ngrok.

Activity 5.1: Local Deployment Preparation

```
python -m venv env
env\Scripts\activate
pip install -r requirements.txt
```

Configure the .env file:

```
GEMINI_API_KEY=your_gemini_api_key_here
NGROK_AUTHTOKEN=your_ngrok_authtoken_here
PORT=5000
```

Activity 5.2: Local Testing

Run the application locally:

```
python app.py
```

Once running, open a browser at http://127.0.0.1:5000 and submit a test startup idea to verify the Viability Score, Innovation Score, SWOT report, and Recommendations are correctly generated, parsed, and rendered end-to-end.

Activity 5.3: Public Deployment via Ngrok

On startup, app.py automatically authenticates with Ngrok using NGROK_AUTHTOKEN and opens a public tunnel to the local server, printing the live URL directly in the terminal:

```
============================================================
 LaunchPad AI is LIVE
 Local:  http://127.0.0.1:5000
 Public: https://opossum-ripping-feminize.ngrok-free.dev
============================================================
```

This public URL is shared with evaluators so they can access the running app directly from their browser without any local installation. Note that on the free Ngrok tier this URL changes on every restart.

Exploring the Website's Web Pages:

Home / Generator Page:

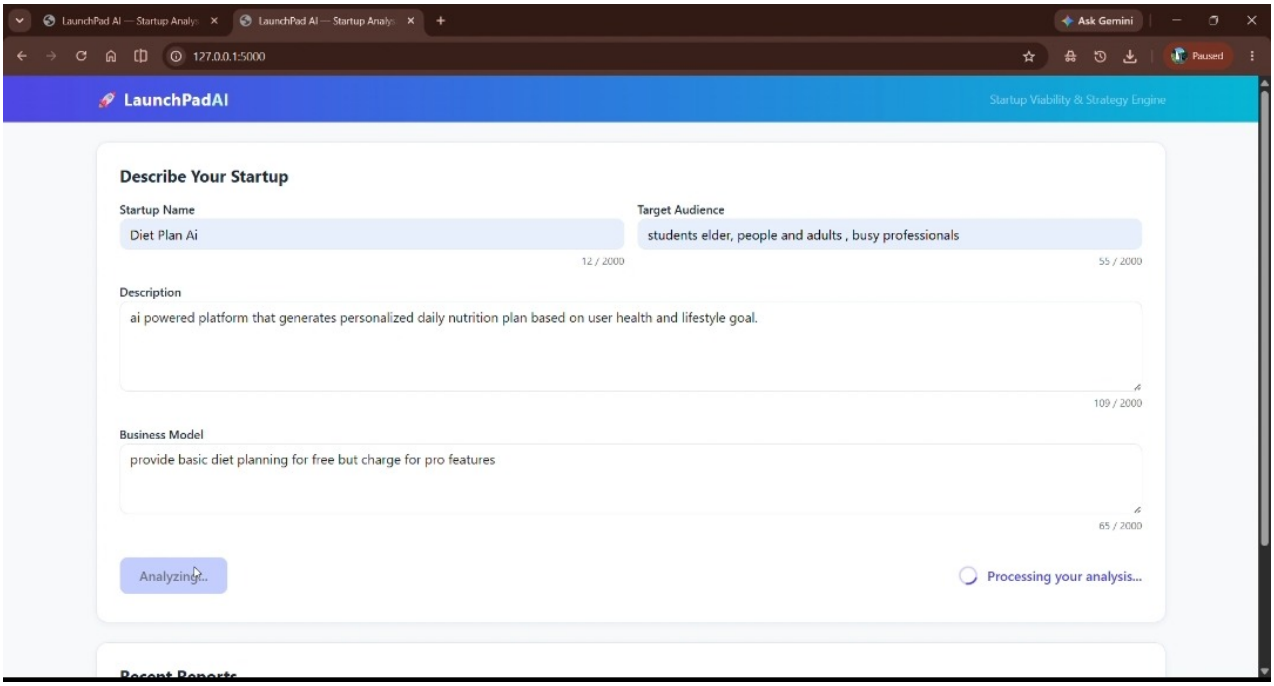

Fig 2: The "Describe Your Startup" input form, filled with a sample idea

Description: The single-page interface serves as both landing page and analysis tool, with a blue-to-teal gradient header reading "LaunchPad AI — Startup Viability & Strategy Engine." The main card contains four labeled fields (Startup Name, Target Audience, Description, Business Model), each with a live 0/2000 character counter, and a primary action button that triggers analysis.

Results Section — Scores & SWOT Report:

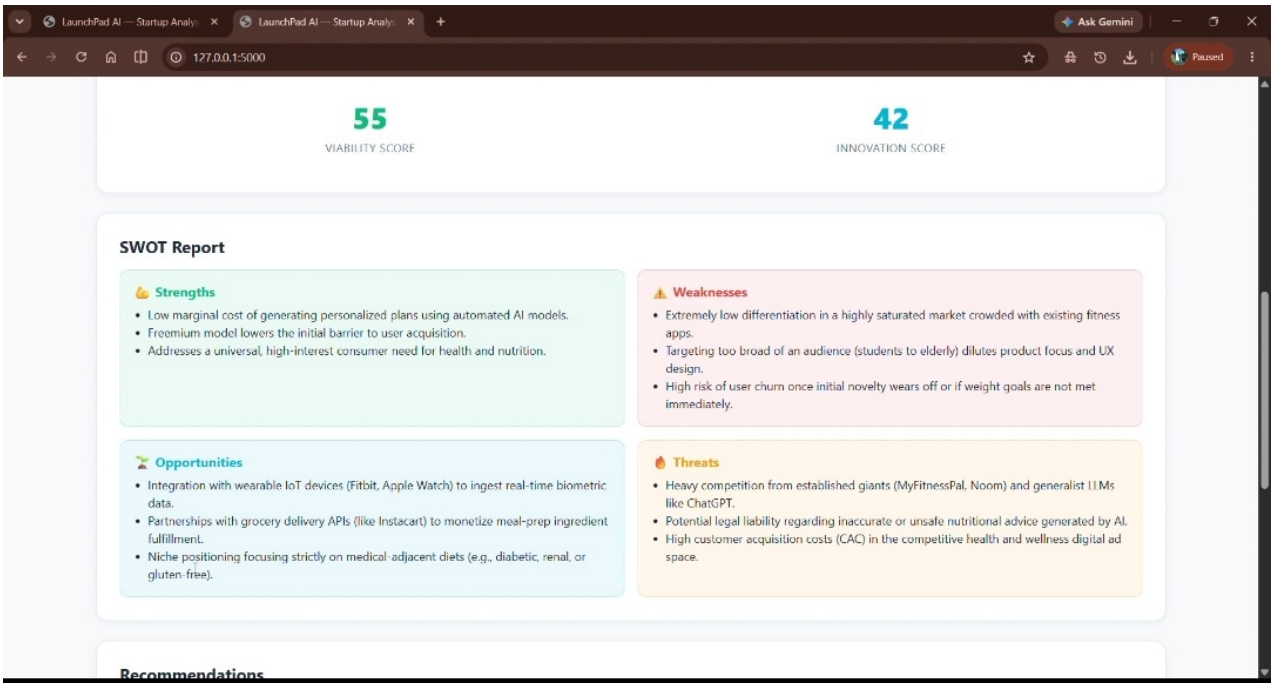

Fig 3: Generated Viability/Innovation scores and SWOT report

Description: Immediately below the form, two large score cards display the Viability Score (55) and Innovation Score (42) in teal and blue. Beneath that, a 2x2 SWOT grid renders Strengths, Weaknesses, Opportunities, and Threats as bulleted cards, each generated fresh from the Gemini response for the submitted idea.

Results Section — Recommendations:

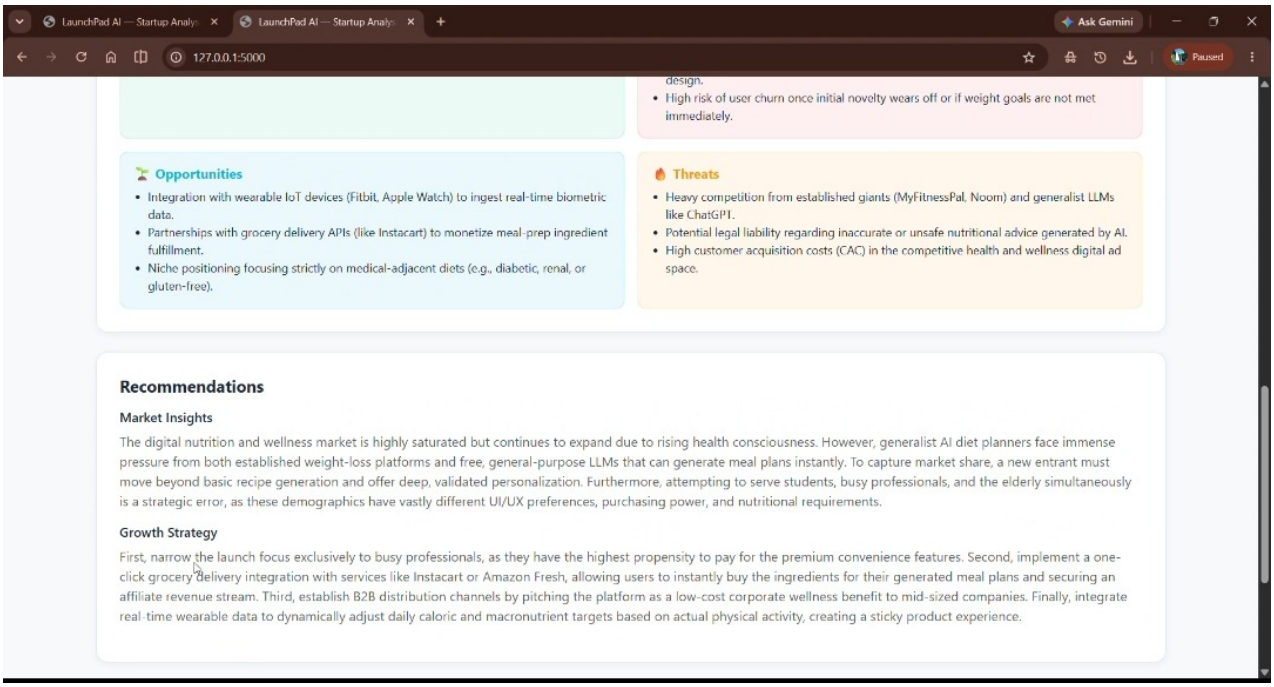

Fig 4: Market Insights and Growth Strategy recommendations

Description: Below the SWOT grid, a Recommendations card presents two free-text sections — Market Insights and Growth Strategy — giving the founder a narrative interpretation of the scores and SWOT points above, followed by a Recent Reports list for quickly revisiting earlier analyses.

Conclusion:

LaunchPad AI is a generative-AI startup analysis platform built with Flask and Flask-SQLAlchemy, using Google's Gemini API (gemini-3.5-flash) to instantly turn a short founder description into a structured Viability Score, Innovation Score, SWOT report, and actionable growth recommendations. The project — covering model selection, a strictly-validated JSON prompt pipeline, a persisted SQLite report history, and public deployment via Ngrok — demonstrates how a constrained, well-validated AI integration can turn an LLM into a dependable structured-analysis tool rather than an unpredictable chat interface. Looking ahead, LaunchPad AI has clear room to grow: exporting reports as shareable PDFs, comparing multiple startup ideas side-by-side, adding user accounts to separate report histories per founder, and fine-tuning the scoring rubric against real startup outcome data. With continued refinement of the underlying prompts and a bit more UI polish, LaunchPad AI is well-positioned to grow from a class project into a genuinely useful early-stage validation tool.